

Tutorato Programmazione 2

Michele Porcellato, Giacomo Vettore

13 Maggio 2026

Istruzioni (leggere attentamente)

Vi sono tre tipi di domande e, di conseguenza, risposte:

1. Un frammento di codice che esegue correttamente. Si indichi l'output nell'apposito riquadro.

Test N l'output è →	
----------------------------	--

2. Un frammento di codice che genera errori. Si indichi in quale riga l'errore avviene; si specifichi la tipologia (in compilazione o a runtime) cerchiando la cella corrispondente (**A** o **B**); si spieghi brevemente la ragione dell'errore. *Tutti e tre i riquadri devono essere compilati.*

errore alla riga n.	A in compilazione	perché →
	B a runtime	

3. Domande vero/falso. Si riporti **V** (vero) o **F** (falso) nelle caselle corrispondenti. In questa tipologia, le *risposte errate sottraggono punti*.

A	B	C	D	E	F	G	H

Test 1

errore alla riga n.	A in compilazione B a runtime	perché →
---------------------	----------------------------------	----------

```
1 public class Test {
2     static final int MAX = 10;
3     public static void main(String[] args) {
4         A a = new A();
5         a.m1(1);
6         C c = new C();
7         c.m2(2);
8     }
9 }
10 interface I1 {
11     public int m1(int x);
12 }
13 interface I2 extends I1 {
14     public int m2(int x);
15 }
16 class A implements I1 {
17     int a = 20;
18     public int m1(int x) {
19         return a + x;
20     }
21 }
22 class B extends A {}
23 class C extends B implements I2 {}
```

Test 2

errore alla riga n.	A in compilazione B a runtime	perché →
---------------------	----------------------------------	----------

```
1 public class Test {
2     static final int MAX = 10;
3     public static void main(String[] args) {
4         A a = new A(10);
5         a.m("mela");
6         A a1 = new C();
7         C c1 = (C) a1;
8         c1.m("pera");
9         A a2 = new B();
10        C c2 = (C) a2;
11        c2.m("banana");
12    }
13 }
```

```

14 interface I1 {
15     public int m(String s);
16 }
17 class A implements I1 {
18     int x;
19     A(int x) { this.x = x; }
20     public int m(String s) {
21         return s.length() * x;
22     }
23 }
24 class B extends A {
25     B() { super(0); }
26 }
27 class C extends B {}

```

Test 3

errore alla riga n.	A in compilazione B a runtime	perché →
---------------------	--	----------

```

1 public class Test {
2     public static void main(String[] args) {
3         A obj = new B();
4         obj.m(new D());
5     }
6 }
7 class A {
8     final void m(C c) { System.out.println("1"); }
9 }
10 class B extends A {
11     void m(C c) { System.out.println("2"); }
12     void m(D c) { System.out.println("3"); }
13 }
14 class C {}
15 class D extends C {}

```

Test 4

Test 4 l'output è →	
----------------------------	--

```

1 public class Test {
2     public static void main(String[] args) {
3         A obj = new B();
4         A par = new B();
5         System.out.println(obj.m(par));
6     }

```

```

7 }
8 class A {
9     int x = 0;
10    String m(A a) { return "a in a"; }
11 }
12 class B extends A {
13    String m(B b) { return "b in b"; }
14    String m(A a) { return "a in b"; }
15 }

```

Test 5

Test 5	l'output è →
--------	--------------

```

1 public class Test {
2     public static void main(String[] args) {
3         A a1 = new A(5);
4         A a2 = new A(3);
5         A a3 = new A(5);
6         System.out.println(a1.equals(a2));
7         System.out.println(!a1.equals(a3));
8     }
9 }
10 class A {
11     int x = 0;
12     A(int x) { this.x = x; }
13 }

```

Test 6

Test 6	l'output è →
--------	--------------

```

1 public class Test {
2     public static void main(String[] args) {
3         A a = new A();
4         int r1 = a.m(6);
5         A a2 = new B(new Z());
6         int r2 = a2.m(1);
7         B b = new B(new Z());
8         int r3 = b.m(2);
9         System.out.println("" + r1 + " " + r2 + " " + r3);
10    }
11 }
12 class Z {}
13 class A {
14     static int val = 5;
15     int m(int x) { val--; return x * val; }

```

```

16 }
17 class B extends A {
18     Z z;
19     B(Z z) { this.z = z; val++; }
20     int m(int x) { return x * val + 1; }
21 }

```

Test 7

Test 7	l'output è →
--------	--------------

```

1 public class Test {
2     public static void main(String[] args) {
3         I i = new C(4);
4         System.out.println(i.m(2));
5     }
6 }
7 interface I {
8     int m(int z);
9 }
10 class A implements I {
11     int x;
12     A(int x) { this.x = x + 1; }
13     public int m(int z) { return x + z; }
14 }
15 class B extends A {
16     B(int x) { super(++x); }
17     public int m(int z) { return x * z; }
18 }
19 class C extends B {
20     C(int x) { super(++x); }
21 }

```

Test 8

Test 8	l'output è →
--------	--------------

```

1 import java.util.*;
2 public class Test {
3     public static void main(String[] args) {
4         String[] a = {"limone", "ananas", "mango", "lime"};
5         Set<Integer> s = new HashSet<>();
6         List<Integer> l = new ArrayList<>();
7         for (String i : a) {
8             s.add(i.length());
9             l.add(i.length());
10        }

```

```

11     System.out.println(s.size() + l.size());
12 }
13 }

```

Test 9

A	B	C	D	E	F	G	H

- A. In una classe possono coesistere più metodi con lo stesso nome ma firme (numero e tipo dei parametri) diverse.
- B. La parola chiave `finally` serve a terminare l'esecuzione del programma.
- C. Si consideri un attributo `x` dichiarato come `protected` nella classe `C` del package `P1`; `x` non è visibile da una classe `D` appartenente a un package `P2`, a meno che `D` non erediti da `C`.
- D. Le istruzioni `Object[] x = new Object[10]; x[0] = "elemento";` sono illegali in Java, perché per gli array non è supportato il polimorfismo; questo è il motivo principale per l'esistenza del Java Collection Framework, che invece supporta questa funzionalità.
- E. Il garbage collector è una funzionalità del supporto run-time di Java molto utile, in quanto gestisce in maniera automatica la deallocazione di oggetti non più utilizzati dal programma, semplificando il lavoro del programmatore ed evitandone potenziali errori.
- F. Quando in Java si parla di un metodo *overloaded* si fa riferimento a un metodo "sovraccarico", cioè la cui implementazione genera una computazione particolarmente pesante.
- G. Una classe Java definita come `abstract` può essere usata all'interno di gerarchie di classi con ereditarietà multipla.
- H. Siano dati due oggetti `a` e `b`, per i quali il test `a.hashCode() == b.hashCode()` ritorna `true`. In questo caso, si può stabilire con certezza che `a` e `b` sono identici, cioè puntano allo stesso oggetto.

Vedi soluzioni a fine documento.

Test 10

Test 10	l'output è →	
---------	--------------	--

```

1 public class Test {
2     public static void main(String[] args){
3         A obj = new B();
4         obj.m(new D());
5     }
6 }
7 class A {
8     void m(C c) { System.out.println("x"); }
9 }
10 class B extends A {
11     void m(C c) { System.out.println("y"); }
12     void m(D d) { System.out.println("z"); }
13 }
14 class C { }
15 class D extends C { }

```

Test 11

errore alla riga n.	A in compilazione B a runtime	perché →
---------------------	--	----------

```

1 import java.util.*;
2 public class Sette {
3     Sette(){
4         List a = new ArrayList();
5         List b = new HashSet();
6         for (int k=0; k<10; k++) {
7             String s = "A" + (k%3);
8             a.add(s);
9             b.add(s);
10        }
11        System.out.println(a.size() + " " + b.size());
12    }
13    public static void main(String[] a) { new Sette(); }
14    public static void main(String a) { new Sette(); new Sette
15    (); }

```

Test 12

Test 12 l'output è →	
-----------------------------	--

```

1 public class Test {
2     public static void main(String[] args) {
3         A a = new B();
4         B b = new B();

```

```

5         J j = b;
6         if (a.equals(b))
7             System.out.println(j.m(1) + b.m("scritto"));
8         else System.out.println(j.m(3));
9     }
10 }
11 interface I { int m(int z); }
12 interface J extends I {}
13 class A implements I {
14     int x = 10;
15     public int m(int z) { return x + z; }
16     public boolean equals(Object o) {
17         A obj = (A) o;
18         return x == obj.x;
19     }
20 }
21 class B extends A implements J {
22     public int m(String s) { return s.length(); }
23 }

```

Test 13

Test 13	l'output è →
---------	--------------

```

1 import java.util.*;
2 public class E6 {
3     static Collection ll = new LinkedList();
4     int x = 6;
5     E6() {}
6     E6(int x) {
7         ll.add(this);
8         ll.add(new EA5());
9     }
10    public static void main(String z[]) {
11        new E6(3);
12        Iterator iter = ll.iterator();
13        while (iter.hasNext()) {
14            ((E6)(iter.next())).f();
15        }
16    }
17    public void f() { System.out.print(x); }
18 }
19 class EA5 extends E6 {
20     public void f() {
21         x++; super.f(); System.out.print(2);
22     }
23 }

```


Test 14

Test 14	l'output è →
---------	--------------

```
1 public class Sei {
2     char f() { return '6'; }
3     public static void main(String e[]) {
4         Sei a = new Sei();
5         Sei b = new Sette();
6         Sette c = new Sette();
7         System.out.print(a.f() + " " + b.f() + " " + c.f() + " "
8             );
9         char ch[] = {'A', 'B', 'A', 'B', 'A', 'B'};
10        int i1 = 0, i2 = 2, i3 = 4;
11        if (a.equals(b)) i1++;
12        if (b.equals(a)) i2++;
13        if (c.equals(b)) i3++;
14        System.out.print(ch[i1] + " " + ch[i2] + " " + ch[i3]);
15    }
16 }
17 class Sette extends Sei {
18     char f() { return '7'; }
19     public boolean equals(Object a) { return (a instanceof Sei); }
20     public int hashCode() { return 3; }
21 }
```

Test 15

Test 15	l'output è →
---------	--------------

```
1 public class Test8 {
2     public static void main(String[] args) {
3         A a1 = new A(5);
4         A a2 = new A(a1.m()/2);
5         boolean b = (a1 == a2);
6         System.out.print(b);
7     }
8 }
9 class A {
10     int x;
11     A(int x) { this.x = x; }
12     int m() { return x*2; }
13 }
```

Test 16

errore alla riga n.	A in compilazione B a runtime	perché →
---------------------	----------------------------------	----------

```
1 import java.util.*;
2 public class B {
3     B() {
4         Collection b = new Collection();
5         for (int k=0; k<10; k++) {
6             String s = "A" + (k%4);
7             b.add(s);
8         }
9         int count = 0;
10        Iterator i = b.iterator();
11        while (i.hasNext()) {
12            Object s = i.next();
13            count++;
14        }
15        System.out.println(count);
16    }
17    public static void main(String[] a) { new B(); new B(); }
18    public static void main(String a)    { new B(); }
19 }
```

Test 17

Test 17	l'output è →
---------	--------------

```
1 public class C {
2     public static int x;
3     C(int s) { x = s; }
4     void f() { System.out.print(x); }
5     public static void main(String a[]) {
6         C b = new C(3);
7         C c = new C(5);
8         b.f();
9         c.f();
10    }
11 }
```

Test 9 Soluzioni

A	B	C	D	E	F	G	H
V	F	V	F	V	F	F	F

- A. In una classe possono coesistere più metodi con lo stesso nome ma firme (numero e tipo dei parametri) diverse.
- B. La parola chiave `finally` serve a terminare l'esecuzione del programma.
- C. Si consideri un attributo `x` dichiarato come `protected` nella classe `C` del package `P1`; `x` non è visibile da una classe `D` appartenente a un package `P2`, a meno che `D` non erediti da `C`.
- D. Le istruzioni `Object[] x = new Object[10]; x[0] = "elemento";` sono illegali in Java, perché per gli array non è supportato il polimorfismo; questo è il motivo principale per l'esistenza del Java Collection Framework, che invece supporta questa funzionalità.
- E. Il garbage collector è una funzionalità del supporto run-time di Java molto utile, in quanto gestisce in maniera automatica la deallocazione di oggetti non più utilizzati dal programma, semplificando il lavoro del programmatore ed evitandone potenziali errori.
- F. Quando in Java si parla di un metodo *overloaded* si fa riferimento a un metodo "sovraccarico", cioè la cui implementazione genera una computazione particolarmente pesante.
- G. Una classe Java definita come `abstract` può essere usata all'interno di gerarchie di classi con ereditarietà multipla.
- H. Siano dati due oggetti `a` e `b`, per i quali il test `a.hashCode() == b.hashCode()` ritorna `true`. In questo caso, si può stabilire con certezza che `a` e `b` sono identici, cioè puntano allo stesso oggetto.